Обеспечение безопасности в Kubernetes

DevOps Advanced

● REC **Проверить, идет ли запись**

Меня хорошо видно & слышно?

Маршрут вебинара

- Управление доступом и конфигурацией
- Безопасность на этапе разработки (Shift-Left)
- Защита и наблюдаемость

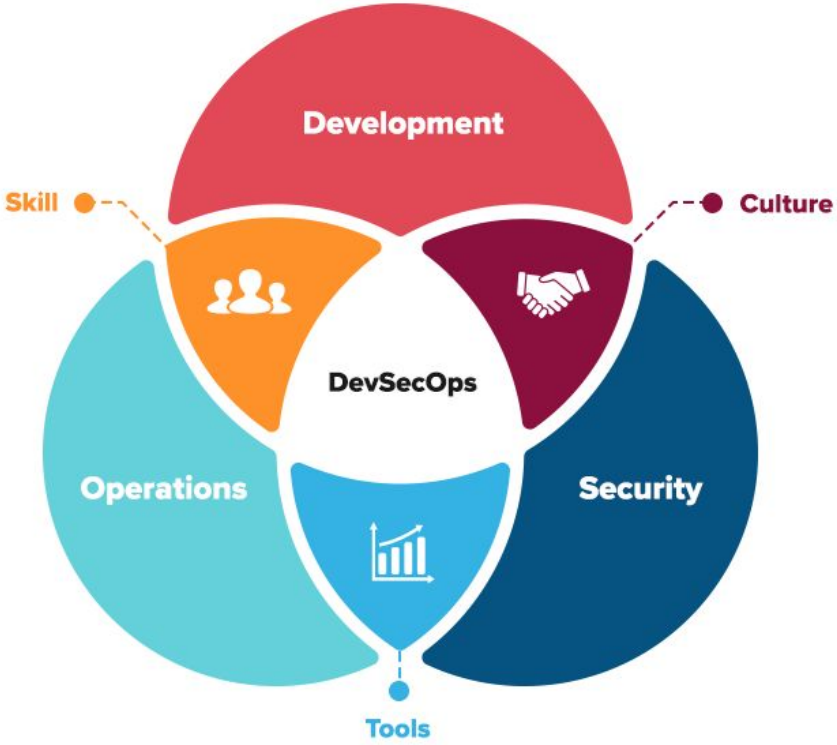
Цели вебинара

К концу занятия вы сможете

настраивать RBAC, NetworkPolicies, PodSecurityPolicies

сканировать образы на уязвимости

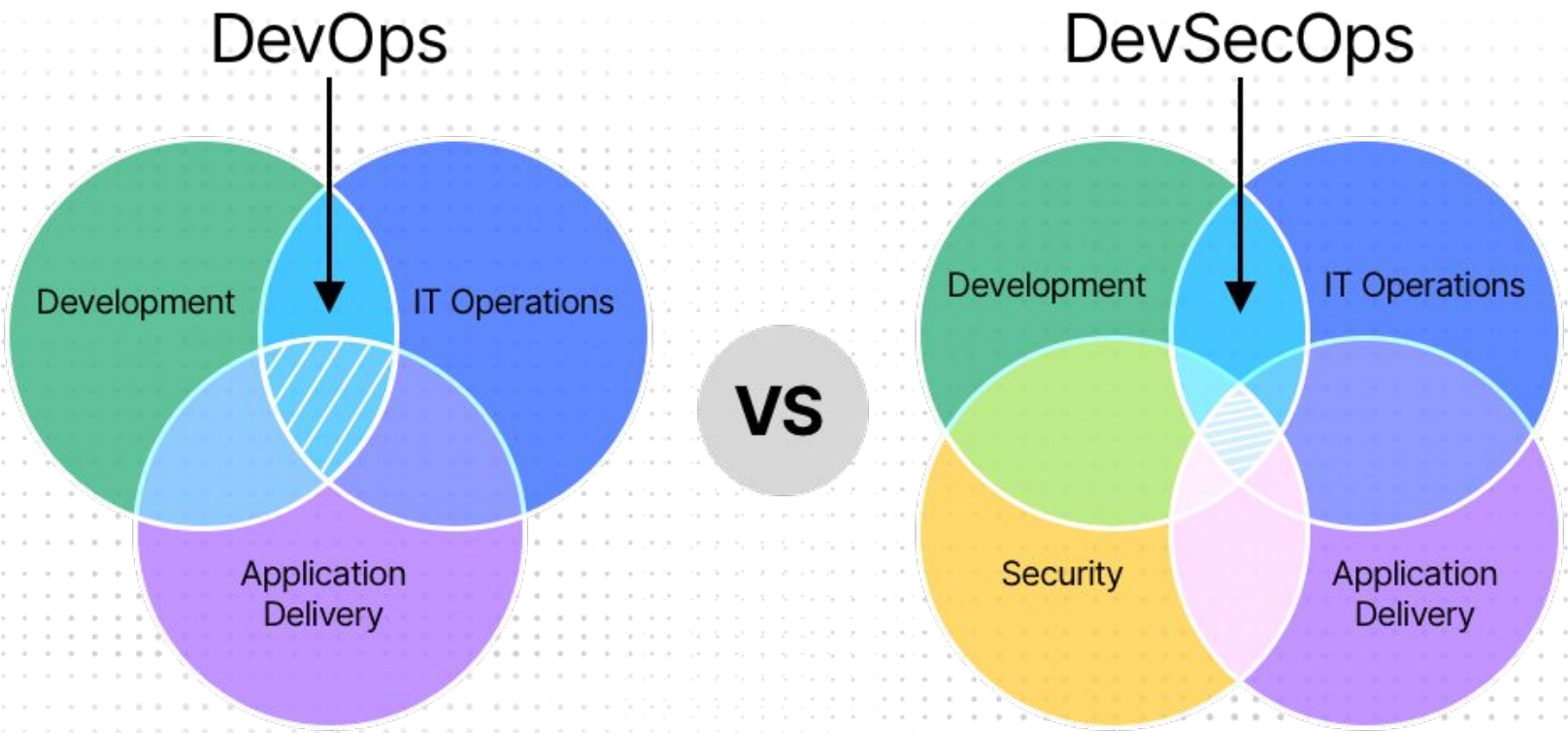
DevSecOps



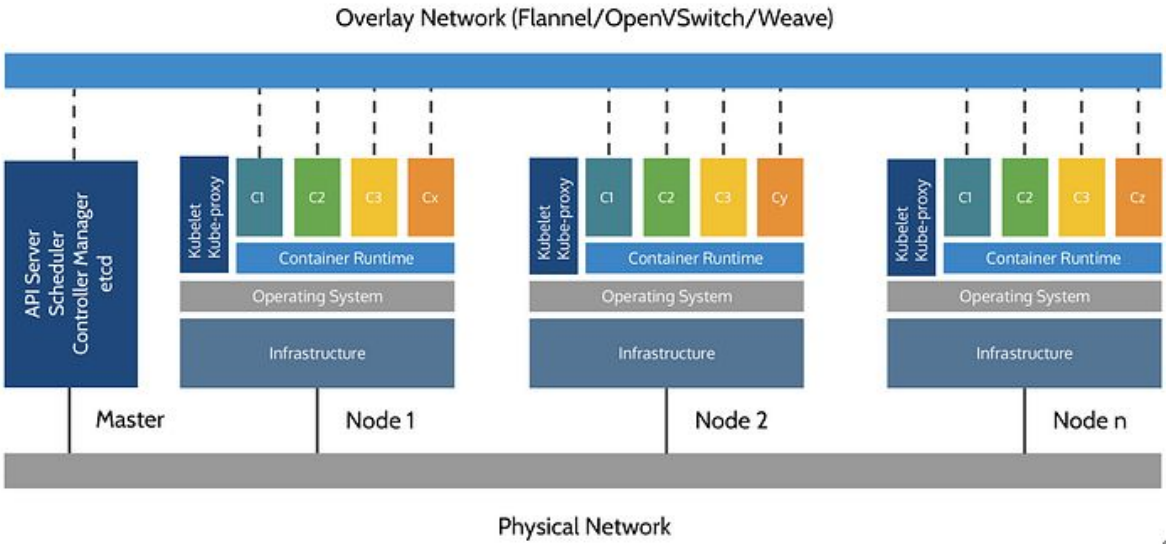
Development
Software releases
and updates

Operations
Reliability Performance
Scaling

Security
Confidentiality Integrity
Availablity


DevOps
Development
IT Operations
Application Delivery
VS
DevSecOps
Development
IT Operations
Security
Application Delivery

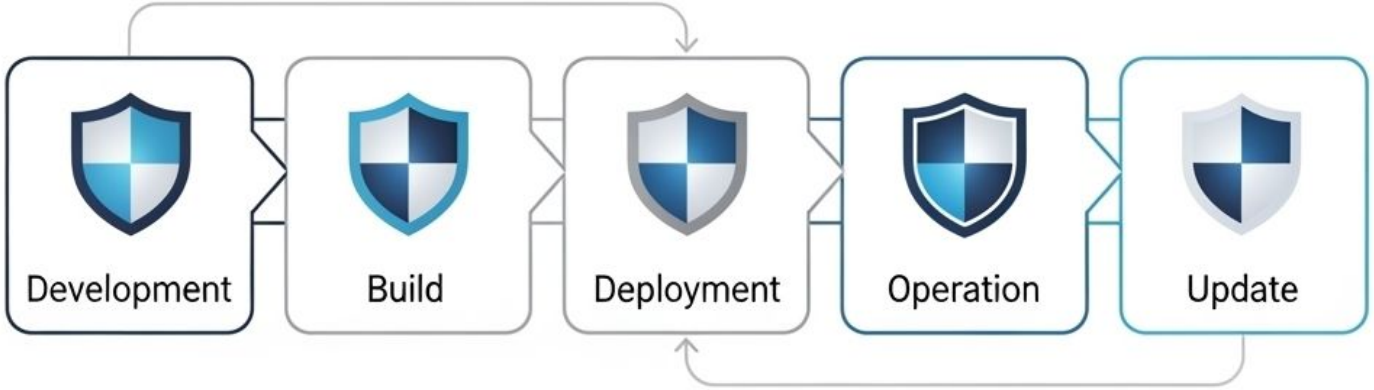
Многоуровневая безопасность: От хоста до приложения



"Defense in Depth: Если один слой пронзен, другие остаются целыми"

Вопрос

- Почему безопасность — это процесс, а не разовое действие?



"Безопасность должна быть встроена (baked in), а не приложена (bolted on)"

Управление доступом и конфигурацией

Kubernetes API Request Flow

kubectl user

Service Account

client-go

API Server

Authentication

Is this a valid user?

Authorization

What can this user do?

Admission Control

Which policies govern this request?

etcd

Node 1

Node 2

Аутентификации и авторизации

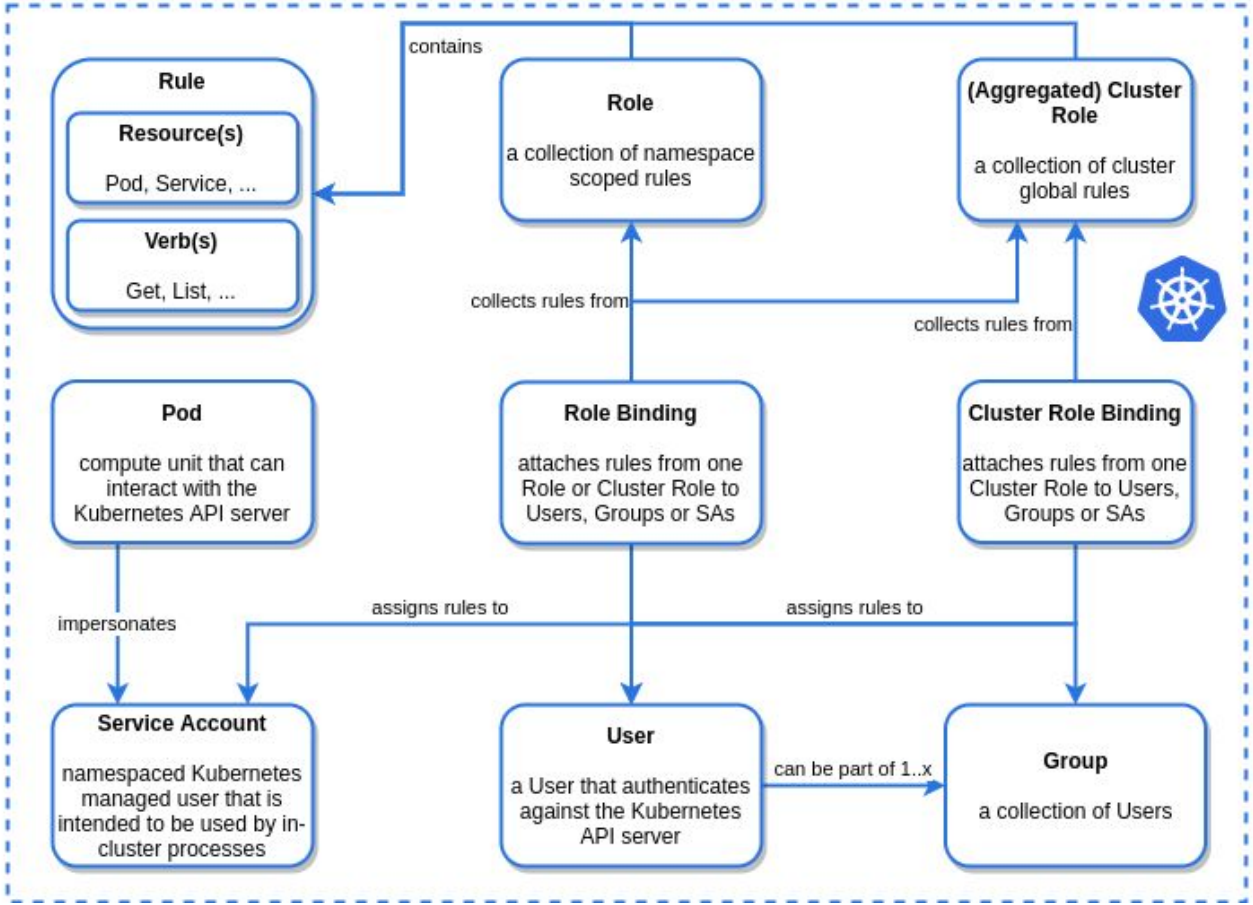
Аутентификация:

- OpenID Connect (OIDC)
- Прокси (Auth Proxy)
- Сервисные аккаунты (SA)
- Имперсонация
- Веб-хук токенов (Webhook Token)

Авторизация:

- Role-Based Access Control (RBAC)
- ABAC (Attribute-Based Access Control)
- Webhook
- Node (kubelet)
- AlwaysAllow / AlwaysDeny

Kubernetes RBAC Objects



RBAC Управление доступом на основе ролей

```
# Основные компоненты:
# `Role` / `ClusterRole`              : Что можно делать (набор разрешений)
# `RoleBinding` / `ClusterRoleBinding`: Кто может это делать (связь с субъектом)

# Строится на:
# Субъекты (subject)  : user, process
# Ресурсы (resources) : pod, nodes, secrets
# Глаголы (verbs)     : create, get, patch, delete, watch

# Базовые роли: admin, edit, view
```

"Принцип наименьших привилегий: давайте только то, что необходимо"

Проверка прав

```
# Синтаксис: `kubectl auth can-i <действие> <ресурс> [--namespace
<имя>]`

# `kubectl auth can-i create pods --namespace default`   → `yes`
# `kubectl auth can-i get secrets --namespace prod`      → `no`
# `kubectl auth can-i update deployments --as dev-team`  → `no`
```

"Безопасность начинается с контроля доступа."

Визуализация прав доступа

```
# Установка : `kubectl krew install who-can`
# Синтаксис : `kubectl who-can <действие> <ресурс> [--namespace <имя>]`

kubectl who-can create pods --namespace production

# USER           SERVICEACCOUNT
# alice@corp.com default:alice-sa
# dev-team       dev:dev-sa_
```

"Безопасность — это прозрачность. Вы должны знать, кто что может делать."

Сетевые политики (NetworkPolicy)

```
# Основные элементы:
# `podSelector` : Группа подов, к которым применяется политика
# `ingress`     : Правила для *входящего* трафика
# `egress`      : Правила для *исходящего* трафика

# `NetworkPolicy` — это механизм для изоляции подов на уровне сети
# По умолчанию все поды могут общаться со всеми
# Политики работают только с CNI-провайдерами
```

“Четкие границы между сервисами, что является основой для модели Zero Trust"

Политики безопасности Pod

```
# Устаревший стандарт : `PodSecurityPolicy (PSP)` — удалена с 1.25

# Современная замена  : `Pod Security Standards (PSS)`
#                     + `Pod Security Admission (PSA)`

# Уровни PSS:
# `privileged` : Минимальные ограничения (не рекомендуется)
# `baseline`   : Блокирует очевидные анти-паттерны
# `restricted` : Наиболее строгий уровень
```

“Migrate from PSP to PSS: Это обязательный шаг для поддержки кластера"

Применение политик безопасности

```
# `Kyverno` : Политики на YAML, простота для типичных задач
# `OPA/Gatekeeper` : Политики на Rego, максимальная гибкость
```

Критерий	Kyverno	OPA/Gatekeeper
Язык для написания политик	**YAML.** Политики пишутся на том же языке, что и манифесты Kubernetes. Это делает их понятными и легко проверяемыми.	**Rego.** Специализированный декларативный язык запросов, основанный на Datalog. Более мощный, но требует изучения.
Сложность освоения	**Низкая.** Идеален для команд, которые уже знакомы с Kubernetes.	**Высокая.** Требует изучения нового языка (Rego) и концепций OPA.
Тип инструмента	**Специализированный движок политик для Kubernetes.** Создан специально для Kubernetes и его API.	**Универсальный движок политик (OPA), адаптированный для Kubernetes (Gatekeeper).** OPA — это общая система управления политиками.

“Автоматизация безопасности: превращаем best practices в железные правила.”

Вопросы

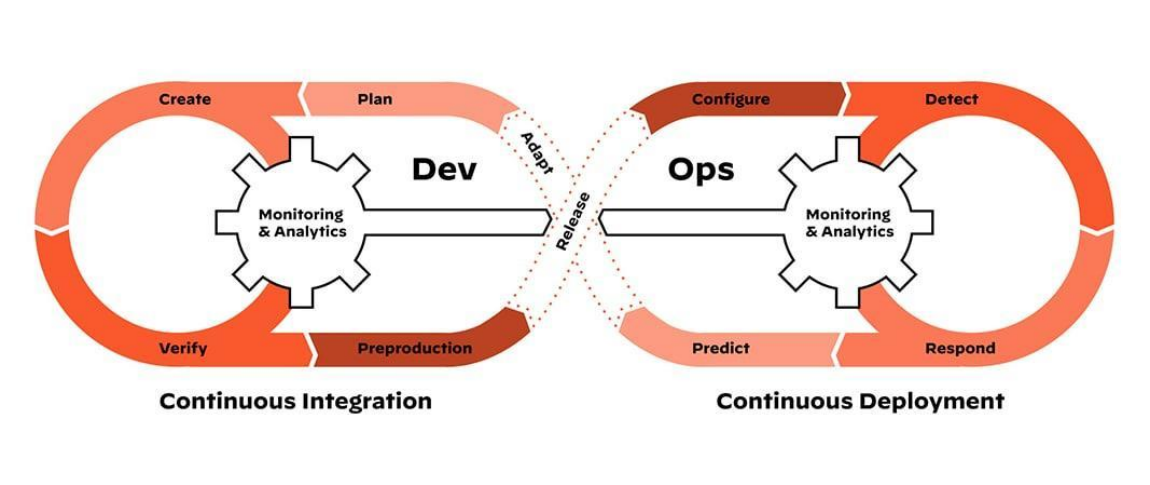
+ если есть вопросы

– если вопросов нет

Безопасность на этапе разработки (Shift-Left)

Сканирование образов (Shift-Left)

```
# `Trivy` (Aqua Security) : Простой, быстрый, проверяет ОС и языковые-пакеты
# `Grype` (Anchore)       : Альтернатива `Trivy`, используется с `Syft` (SBOM)
```



"Найди уязвимость в CI — спаси production от инцидента"

Проверка манифестов

```
# Цель: Автоматизировать проверку манифестов на соответствие best practices
        и выявление анти-паттернов

# `kube-score` : CLI-инструмент, простота, идеален для CI/CD
# `polaris`    : Веб-интерфейс, оценка безопасности (например, 95%), 70+ проверок
# `conftest`   : Пишите кастомные политики на языке Rego для глубокой проверки
```

"Лучше получить ошибку в CI, чем инцидент в production."

Цепочка поставок (Supply Chain Security)

```
# Ключевые компоненты:
# SBOM (Software Bill of Materials): "Ведомость материалов" для ПО
# `Syft`: Инструмент для генерации SBOM из Docker-образа
# `cosign`: Инструмент для подписи образов, гарантирующий их подлинность и целостность

# Практический пайплайн:
1. Сборка образа
2. `syft my-registry/my-app:v1.0 > sbom.cdx` (Генерация SBOM)
3. `cosign sign --key cosign.key my-registry/my-app:v1.0` (Подпись образа)
4. Развертывание (с проверкой подписи, например, через `Kyverno`)
```

"Доверяйте, но проверяйте. Подтверждайте подлинность и целостность артефактов."

Trivy + Checkov + GitLeaks

Критерий	Trivy	Checkov	GitLeaks
Основная цель	Сканирование образов на уязвимости (CVE)	Проверка IaC (манифесты, Terraform) на анти-паттерны	Поиск секретов (паролей, токенов) в Git
Что проверяет	ОС-пакеты, языковые зависимости	YAML/JSON манифесты Kubernetes	Контент файлов в репозитории
Пример проблемы	Образ содержит openssl с уязвимостью CVE-2023-1234	Манифест не имеет resources.limits	Файл config.json содержит "password": "mysecretpassword"

"Безопасность — это не один инструмент, а экосистема проверок."

Вопросы

+ если есть вопросы

– если вопросов нет

Защита и наблюдаемость

Аудит безопасности кластера: `kube-bench`

```
# Что это: Инструмент автоматизации аудита конфигурации кластера на соответствие CIS Benchmarks
# Как работает: Запускается на узлах и проверяет конфигурационные файлы и параметры компонентов
# Результат: Генерирует отчет с пометками `PASS`, `FAIL`, `WARN`, `INFO`

# Запуск на узле
.\ docker run --rm -v /etc:/etc:ro -v /var:/var:ro -t aquasec/kube-bench:latest

# Запуск для control plane
docker run --rm -v /etc:/etc:ro -v /var:/var:ro -t aquasec/kube-bench:latest --targets master
```

" Безопасность — это не разовое действие. Регулярный аудит — это ключ к стабильности. "

Защита секретов: `Sealed Secrets`, `HashiCorp Vault`

```
# `Sealed Secrets` (Bitnami): для простоты и GitOps
# Шифрует `Secrets` локально в `SealedSecret`
# `SealedSecret` можно безопасно коммитить в Git
# Расшифровывается только в кластере контроллером

# `HashiCorp Vault`: для масштаба и сложных сценариев
# Централизованное хранилище секретов.
# Динамические секреты, ограниченное время жизни
# Интеграция через `CSI Driver` или `Vault Agent`
```

"Ни в коем случае не храните пароли в открытом виде в коде."

Безопасность на уровне ядра eBPF и inspektor-gadget

```
# Возможности:
# Трассировка системных вызовов (какой процесс что делает)
# Мониторинг сетевых соединений и файловых операций
# Профилирование потребления ресурсов

# Трассировка открытия файлов всеми подами в namespace 'my-app’
.\ kubectl gadget trace open -n my-app
```

"eBPF позволяет видеть не только 'что разрешено’."

kube-bench vs inspektor-gadget

Критерий	kube-bench	inspektor-gadget
Философия	Безопасность через соответствие стандартам	Безопасность через наблюдение за поведением
Метод	Статический аудит конфигурации	Динамическая трассировка в реальном времени
Проверяет	Файлы конфигурации, флаги запуска (API-сервер, etcd)	Системные вызовы, сетевые соединения, операции с файлами
Аналогия	Пожарная инспекция: "Есть ли огнетушитель?"	Система видеонаблюдения: "Кто пытается взломать сейф?"
Сила	Гарантирует базовую "гигиену безопасности"	Обнаруживает аномалии и атаки, не подпадающие под правила
Слабость	Не видит активные атаки после настройки	Требует глубоких знаний для интерпретации данных

"Идеальный сценарий: `kube-bench` для аудита, `inspektor-gadget` для мониторинга и диагностики."

Аудит безопасности кластера `ksniff-tcpdump` vs `inspektor-gadget`

Критерий	Традиционные инструменты (ksniff/tcpdump)	eBPF (inspektor-gadget)
Уровень диагностики	Уровень сетевых пакетов (L3/L4)	Уровень ядра и системных вызовов (L0-L2)
Технология	Захват пакетов с помощью tcpdump	Выполнение безопасного кода в ядре (eBPF)
Что показывает	Содержимое IP-пакетов, TCP-флаги, DNS-запросы	Какое приложение открыло файл, какой процесс установил соединение, какие системные вызовы были сделаны
Производительность (Overhead)	Может быть высоким при большом объеме трафика	Очень низкий, так как eBPF-программы эффективны
Требования к правам	Требуются права на запуск tcpdump в контейнере	Требуются права на выполнение eBPF-программ (обычно кластер-админ)
Использование в production	Осторожно, только для краткосрочной диагностики	Безопаснее, так как не создает больших объемов данных

"Выбор инструмента зависит от задачи: захват пакетов vs глубокая интроспекция."

Вопросы

+ если есть вопросы

– если вопросов нет

От защиты к автономной безопасности

Тренды 2025-2026

```
# Zero Trust (Ноль доверия):
# Принцип: "Никому и ничему нельзя доверять по умолчанию“
# Реализация: Service Mesh (Istio) с mTLS для всех вызовов

# AIOps (Искусственный интеллект для эксплуатации):
# Использование LLM для обнаружения аномалий, RCA и предсказания сбоев
# Автоматизация отладки и исправления

# eBPF Everywhere:
# eBPF как фундамент для безопасности, сетей и наблюдаемости
# Инструменты: Cilium, Hubble, inspektor-gadget
```

"Будущее безопасности — в автоматизации и автономности."

AI-Driven Security

```
# Проблема: Объем данных (логи, метрики, события) слишком велик для человека

# Решение: Использование LLM и машинного обучения для:

# Обнаружение аномалий: Выявление подозрительного поведения,
#                        не попадающего под правила

# Определение корневой причины (RCA): Автоматический анализ цепочки событий

# Генерация отчетов: Создание понятных, контекстных инсайтов на естественном языке
```

"Будущее безопасности — в понимании, а не только в защите."

Заключение

```
# Безопасность – это непрерывный процесс, а не разовое действие

# От реактивной диагностики к проактивной и автономной защите

# Эволюция: от конфигурации к поведению (eBPF) и к ИИ (AIOps)
```

"Спасибо за внимание! Ваши вопросы?"

Вопросы

+ если есть вопросы

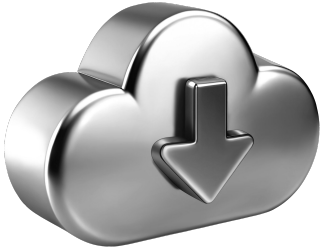
– если вопросов нет

Дополнительные материалы

Ресурсы и дальнейшее обучение

1. Лапаз Рауль — *"Изучаем безопасность Kubernetes, 2-е издание"* (2025)
2. Бранд А., Ландер Р., Россо Д., Харрис Д. — *"Kubernetes на практике"* (2022)
3. Стронг Дж., Лэнси В. — *"Kubernetes и сети"* (2024)
4. Вьяс Дж., Лэнси В. — *"Kubernetes изнутри"* (2022)
5. Уильям Д. — *"Kubernetes для разработчиков (Для профессионалов)"* (2025)
6. Кнаббен А., Фенвик М. и др. — *"Kubernetes. Идеи и рецепты"* (2022)

GitHub: Рекомендуемая литература.md



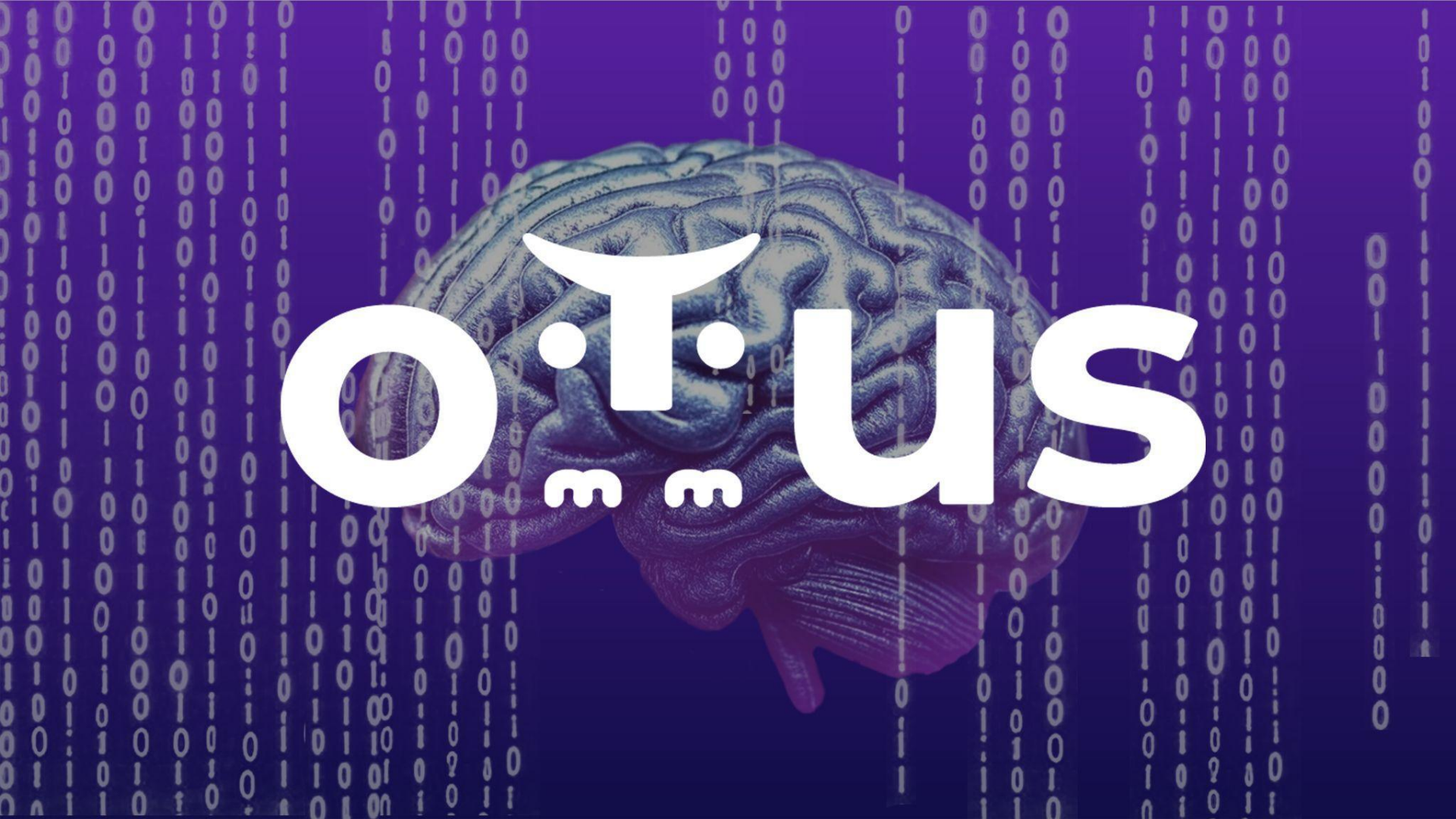
Вопросы

+ если есть вопросы

– если вопросов нет

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️


otus